

Embedded Fire DetectorPi - Agile Documentation

Giacomo Tortora	0522501927
Andrea Bucci	0522501969

Sommario

I.	Vision and User Stories	3
	User stories	3
II.	Technical Architecture	4
	Software Stack & Layers	4
III.	Hardware Map	4
IV.	Dependencies.....	5
V.	Security	6

Vision and User Stories

Develop a Raspberry Pi-based fire monitoring system capable of identifying outbreaks in real time using computer vision. The goal is to minimize response times through immediate hardware response (local alarm) and timely communication with IoT infrastructures (MQTT), while ensuring streamlined, cost-effective, and easily scalable installation in industrial or residential environments.

User stories

Story 1: Critical Monitoring

As a security manager, I want the system to process video continuously and seamlessly (real-time), so that any potential fire outbreak is detected instantly without delays due to lag or software interruptions.

Story 2: Instant Alarm

As a plant operator, I want an audible buzzer to sound instantly when flames are detected, so I can evacuate the area or intervene before the fire spreads.

Story 3: Remote Integration

As an IT administrator, I want to receive alarm data via MQTT to a central server so I can log the event and automatically notify the fire department.

Technical Architecture

The system follows an Edge AI architecture, where processing occurs entirely on the local device without relying on the cloud for model inference.

Software Stack & Layers



*The system is structured into five functional modules, following the **Single Responsibility Principle**. This ensures that each class manages a unique aspect of the fire detection lifecycle:*

- **FrameProvider:** Responsible for the data acquisition layer. It manages the connection to the video source (camera or local file) and provides a continuous stream of raw frames, abstracting the complexity of the video capture backend.
- **ImageProcessor:** Acts as the data preparation layer. It performs essential preprocessing tasks on the raw frames, such as resizing, color space conversion, and normalization, to ensure the input data matches the exact requirements of the neural network.
- **InferenceEngine:** The core computational module. It encapsulates the **TensorFlow Lite Interpreter**, handling the loading of the .tflite model and executing the mathematical operations (inference) to calculate the probability of fire in each processed frame.
- **AlertNotifier:** Manages the multi-channel output logic. It controls the **GPIO hardware triggers** for the buzzer and orchestrates the **MQTT messaging**. It includes internal state-tracking to ensure that notifications and sounds are managed logically, preventing redundant triggers.
- **FireMonitor:** The central **Orchestrator** of the application. It manages the main execution loop, synchronizing the flow of data between the other four modules. It receives frames from the provider, sends them through the processor and engine, and finally instructs the notifier based on the detection results.

Hardware Map

3v3 Power	1	2	5v Power
GPIO 2 (I2C1 SDA)	3	4	5v Power
GPIO 3 (I2C1 SCL)	5	6	Ground BUZZER GND
GPIO 4 (GPCLK0)	7	8	GPIO 14 (UART0 TX)
Ground	9	10	GPIO 15 (UART0 RX)
GPIO 17 BUZZER +	11	12	GPIO 18 (PCM CLK)
GPIO 27	13	14	Ground
GPIO 22	15	16	GPIO 23
3v3 Power	17	18	GPIO 24
GPIO 10 (SPI0 MOSI)	19	20	Ground
GPIO 9 (SPI0 MISO)	21	22	GPIO 25
GPIO 11 (SPI0 SCLK)	23	24	GPIO 8 (SPI0 CE0)
Ground	25	26	GPIO 7 (SPI0 CE1)
GPIO 0 (EEPROM SDA)	27	28	GPIO 1 (EEPROM SCL)
GPIO 5	29	30	Ground
GPIO 6	31	32	GPIO 12 (PWM0)
GPIO 13 (PWM1)	33	34	Ground
GPIO 19 (PCM FS)	35	36	GPIO 16
GPIO 26	37	38	GPIO 20 (PCM DIN)
Ground	39	40	GPIO 21 (PCM DOUT)

Dependencies

To ensure high performance and reliability on an edge computing device, the system leverages a set of specialized open-source libraries. These dependencies handle everything from low-level hardware control to high-level computer vision and communication.

Core Libraries and Frameworks

-  • **TensorFlow Lite Runtime (tflite-runtime):** This is the core AI engine. It is a lightweight version of the TensorFlow framework, specifically optimized for embedded devices like the Raspberry Pi. It allows the system to run the .tflite model with low latency and minimal memory footprint.
- **OpenCV (opencv-python):** Used for all image processing and video handling tasks. It provides the tools for capturing video streams, resizing frames, and overlaying real-time detection data (visual feedback) on the output.
- **NumPy:** A fundamental library for scientific computing. In this project, it is used to manage and manipulate the video frames as multi-dimensional arrays, which is essential for the data format required by the AI model.
- **PyCryptodome:** This library provides the cryptographic primitives used to secure the system. It implements the **Advanced Encryption Standard (AES)** in **CBC (Cipher Block Chaining)** mode. By utilizing a **256-bit key**, it ensures that fire detection alerts are encrypted before transmission, protecting data integrity and preventing unauthorized parties from intercepting or tampering with the telemetry signals.
- **Paho-MQTT:** A light-weight client library that implements the **MQTT (Message Queuing Telemetry Transport)** protocol. It manages the asynchronous communication between the Raspberry Pi and the local broker. It ensures reliable message delivery using a publish/subscribe architecture, allowing the encrypted data to reach the Node-RED orchestration layer in real-time.
- **RPi.GPIO:** This library provides the interface to the Raspberry Pi's General Purpose Input/Output (GPIO) pins. It is used by the AlertNotifier class to control the physical buzzer by sending digital signals to the designated hardware pins.

Execution Environment

-  • **Python Virtual Environment (venv):** All dependencies are managed within an isolated virtual environment. This ensures that the system remains stable and avoids conflicts with the global operating system libraries, making the project easily portable and reproducible.

Security

In this application context, security is not limited to network protection but extends to **Model Robustness** against evasion attacks and environmental noise. The system has been engineered to maintain high reliability even in suboptimal conditions.

A common threat to AI systems is the **Evasion Attack**, where input data is perturbed (e.g., through blur or noise) to cause misclassification. This project addresses this vulnerability through several inherent and architectural factors:

- **Diverse Dataset Training:** The underlying model was trained on an extensive dataset that included a wide variety of "noisy" images. This pre-existing knowledge allows the model to generalize effectively even when the input is not crystal clear.
- **Resolution Rescaling Effects:** To optimize performance for the Raspberry Pi hardware, high-definition video frames are rescaled. This process naturally introduces a level of quality loss similar to noise. However, since the model was trained to be resilient, it maintains high precision despite this degradation, as well as in the presence of blur or extreme lighting conditions.
- **Temporal Smoothing and Probability Averaging:** To prevent erratic behavior and mitigate the impact of transient "adversarial" frames or sensor glitches, a **Temporal Smoothing** logic has been implemented within the FireMonitor orchestrator. Instead of triggering an alarm based on a single frame's inference, the system calculates a **Moving Average** of the confidence scores. The system maintains a **Smoothing Buffer** using a double-ended queue (deque) with a window size of 5 frames. This logic is fundamental for security because it filters out **outliers**. For example, if a single frame is heavily blurred or corrupted (dropping the probability from 70% to 1%), the average remains stable. Since fire is a physical phenomenon that persists over time, requiring consistent detection over 5 frames significantly increases the system's **robustness against false negatives and temporary sensor interference**.

In addition, to ensure maximum confidentiality and protection against replay attacks, all telemetry data is encrypted using **AES-256 in CBC mode**. For every transmission, the system generates a unique **16-byte Initialization Vector (IV)**, which is prepended to the encrypted payload. This ensures that identical sensor readings (e.g., multiple 'FIRE_DETECTED' events) result in entirely different encrypted strings. Each JSON payload is padded according to **PKCS7 (Public-Key Cryptography Standards #7)** standards to satisfy block size requirements. Upon reception, the Node-RED orchestration layer utilizes the **CryptoJS** library to extract the IV, decrypt the stream, and restore the original JSON data for real-time dashboard visualization.
